HEIG-VD / INGÉNIERIE DES MÉDIAS / PROGRAMMATION WEB

OOJS: INTRO + PROTOTYPES

```
const user = {
  user: "simon",
  password: "1234",
  login(password) {
    // login logic
  },
  sendMessage(str) {
    // sending logic
  }
};
```

La programmation orientée objet (POO) est un **paradigme** de programmation basé sur le concept d'objets.

On utilise des objets pour **modéliser** des caractéristiques concrètes (un utilisateur) ou abstraites (un élément html)

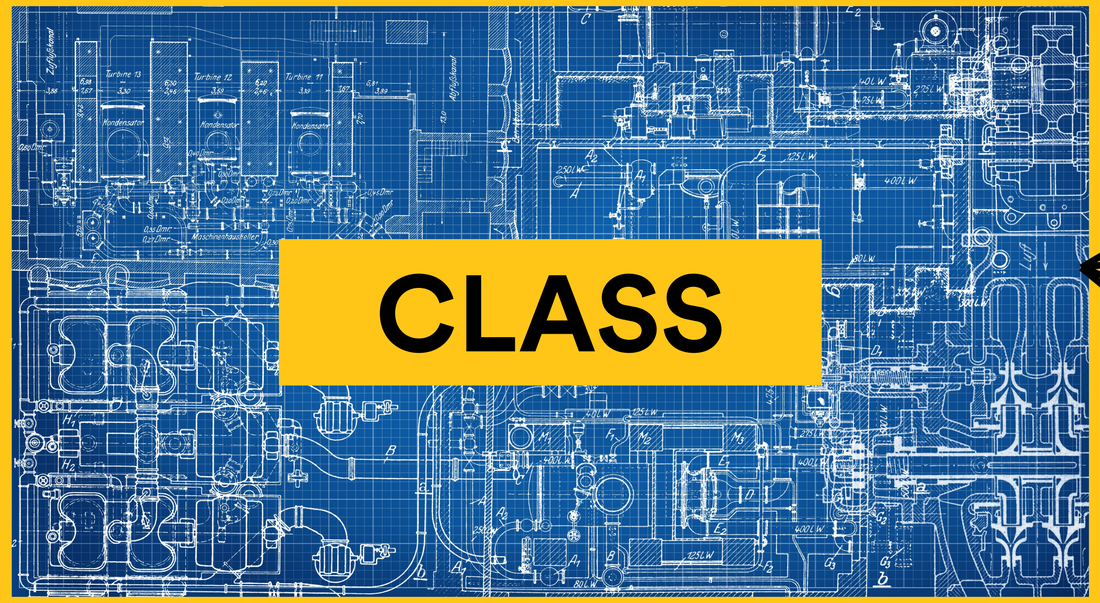
Un objet peut contenir des **données**, et des **méthodes**.

En POO, chaque objet est un bout de code **indépendant**.

Des objets peuvent interagir les uns avec les autres grâce à une **interface** (API).

HE IG VD





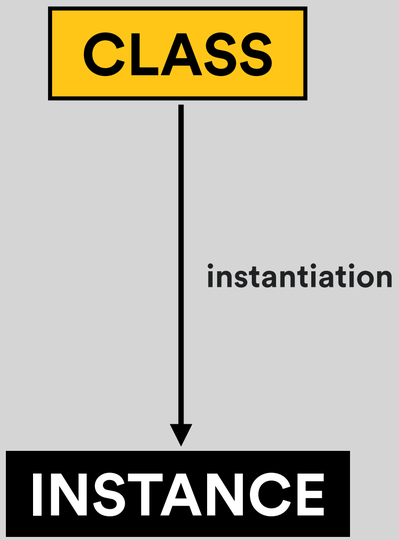
INSTANCE

INSTANCE

INSTANCE

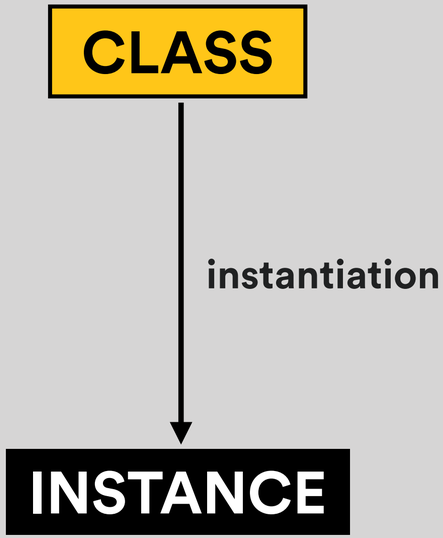
INSTANCE

POO Classique: Classes



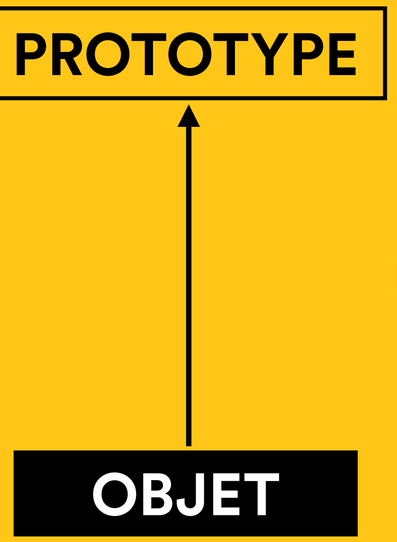
Les objets sont des instances d’une classe.

POO Classique: Classes



Les objets sont des instances d’une classe.

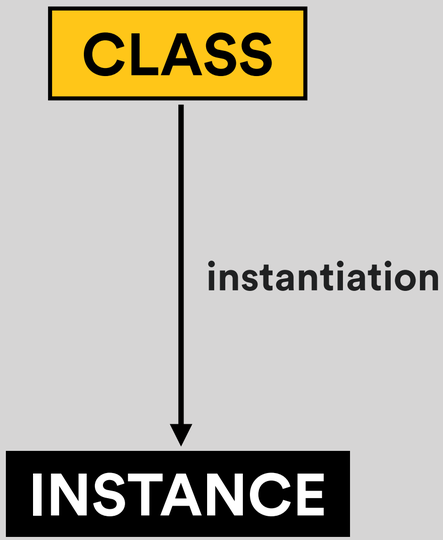
POO en JavaScript: Prototypes



Les objets sont liés à un prototype.

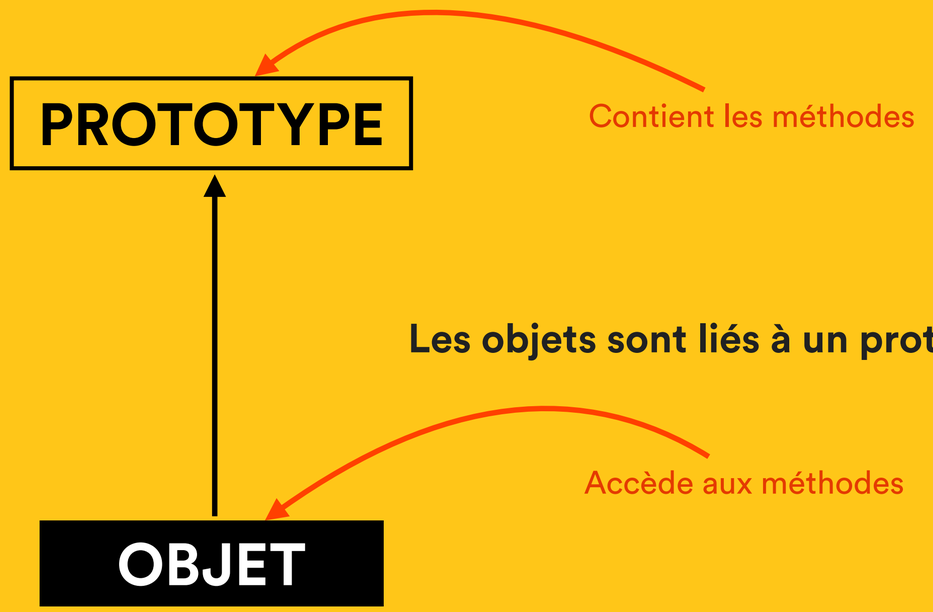
HE VD IG

POO Classique: Classes



Les objets sont des instances d'une classe.

POO en JavaScript: Prototypes

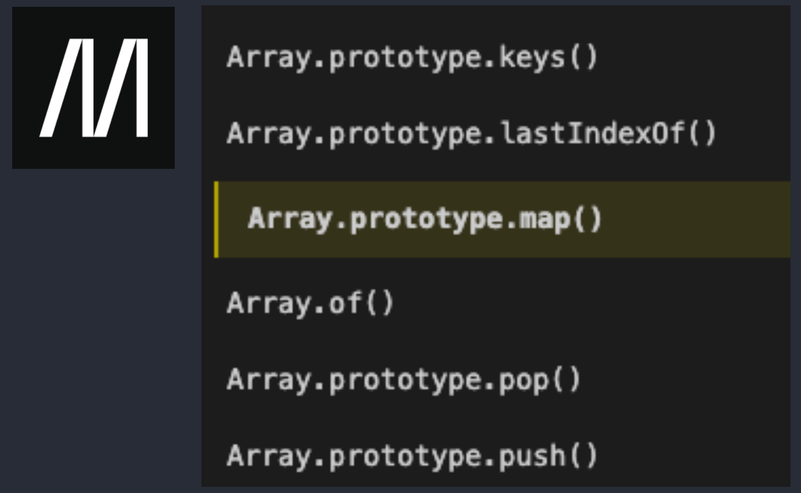


Héritage par prototype: Le prototype contient des méthodes qui sont accessibles à tous les objets liés à ce prototype.

Les méthodes sont déléguées au prototype!

HE IG VD

```
const nums = [1, 2, 3];
nums.map((n) ⇒ n * 2);
```



`Array.prototype` est le prototype de tous les objets array que nous créons en JavaScript.

En conséquence, tous les tableaux ont accès à la méthode `map()`



Comment créer des prototypes? Comment lier des objets à des prototypes? Comment pouvons-nous créer des nouveaux objets sans avoir de classe?

HE VD
IG

1 Constructor functions

👉 Technique nous permettant de créer des objets à partir d’une fonction

👉 C’est comme ça que les objets tels que Array, Map et Set sont implémentés.

1 Constructor functions

👉 Technique nous permettant de créer des objets à partir d'une fonction

👉 C'est comme ça que les objets tels que Array, Map et Set sont implémentés.

2 Classes ES6

👉 Une alternative contemporaine aux constructor functions.

👉 "Sucre syntaxique": dans les coulisses, le classes ES6 utilisent les prototypes.

👉 Les classes ES6 ne se comportent pas comme les classes classiques.

1 Constructor functions

- 👉 Technique nous permettant de créer des objets à partir d'une fonction
- 👉 C'est comme ça que les objets tels que Array, Map et Set sont implémentés.

2 Classes ES6

- 👉 Une alternative contemporaine aux constructor functions.
- 👉 "Sucre syntaxique": dans les coulisses, le classes ES6 utilisent les prototypes.
- 👉 Les classes ES6 ne se comportent pas comme les classes classiques.

3 `Object.create()`

- 👉 La façon la plus simple de lier un objet à un prototype.

HE IG VD

```
const Person = function (name, year) {
  this.name = name;
  this.year = year;
};

new Person("Anna", 1985);
```

Une fonction constructor, est une fonction normale, appelée avec le mot-clé `new`.

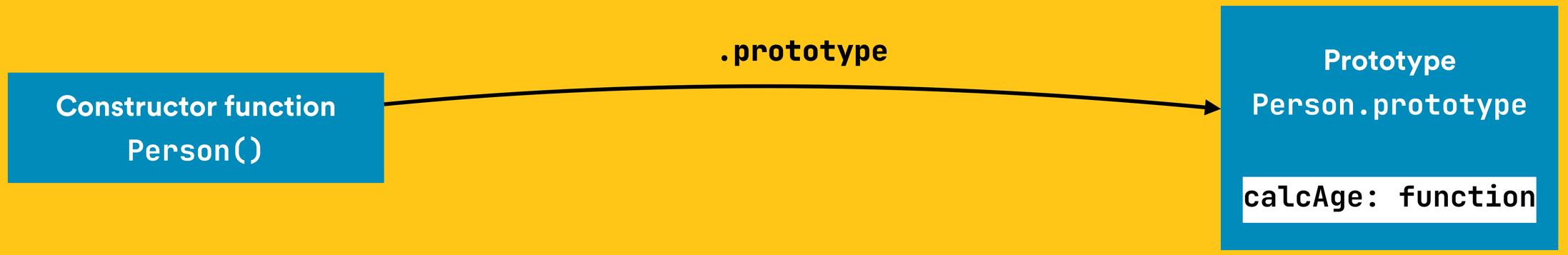
1. Un nouvel objet est créé.
2. La fonction est appelée, `this` se réfère à présent à l'objet.
3. Le nouvel objet est automatiquement lié à un prototype.
4. La fonction retourne automatiquement l'objet.

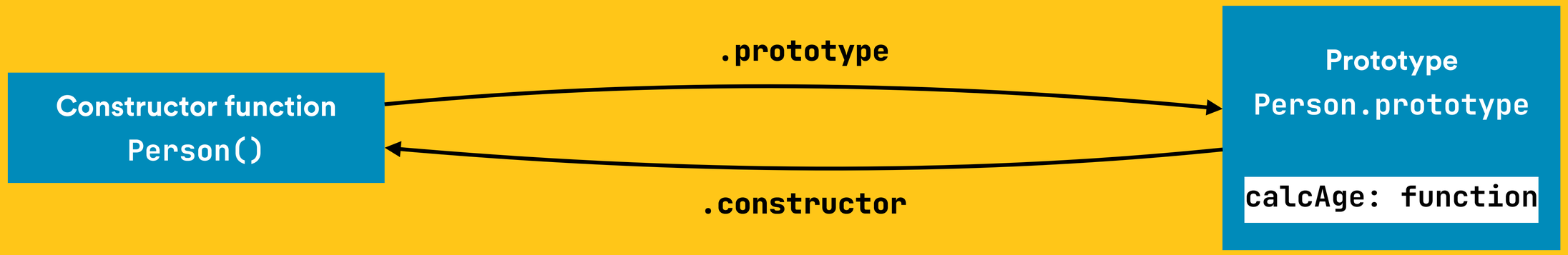
⛔ Attention une fonction constructor ne peut pas être fléchée.

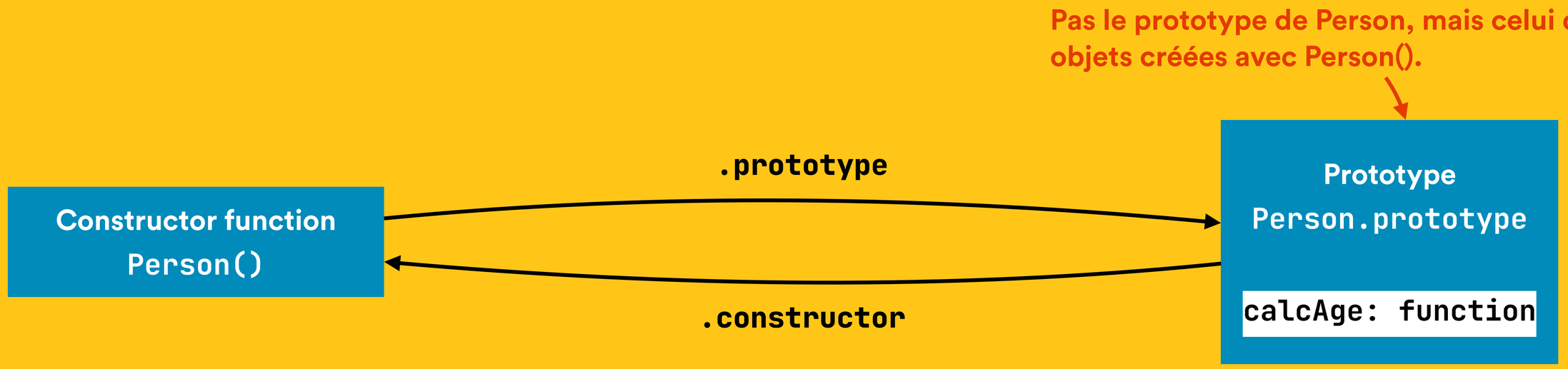
HEIG VD

Constructor function

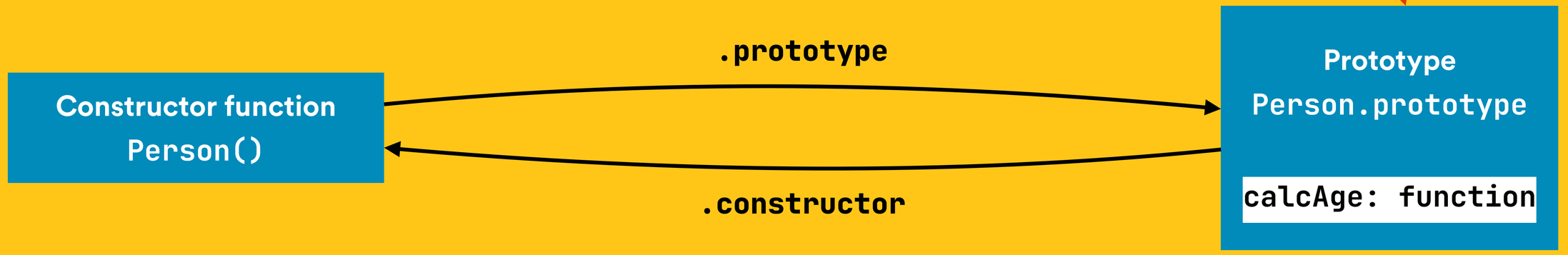
`Person()`


Constructor function
Person()
.prototype
Prototype
Person.prototype
calcAge: function


Constructor function
Person()
.prototype
.constructor
Prototype
Person.prototype
calcAge: function

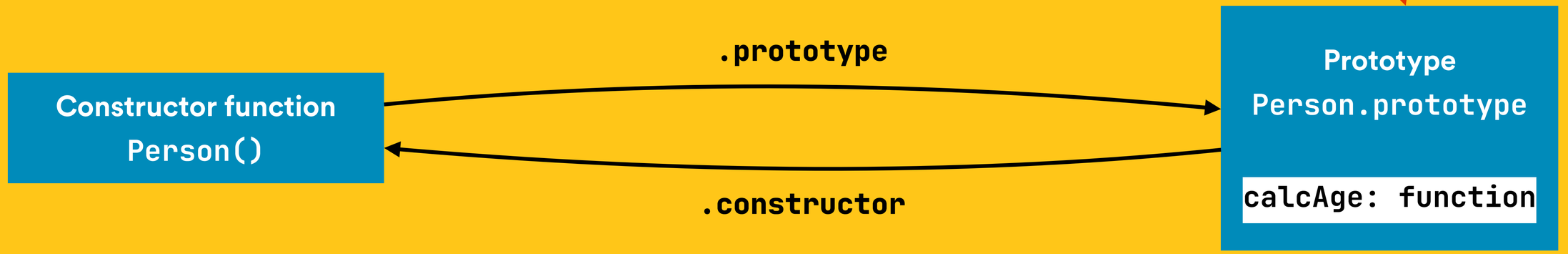

Pas le prototype de Person, mais celui des objets créées avec Person().
Constructor function
Person()
.prototype
.constructor
Prototype
Person.prototype
calcAge: function

Pas le prototype de Person, mais celui des objets créées avec Person().

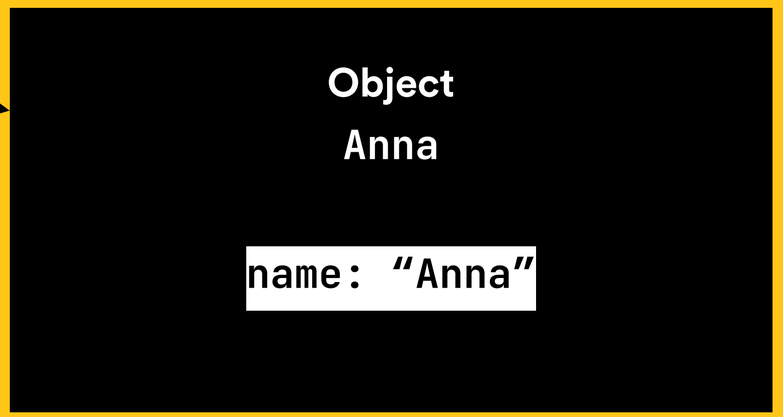


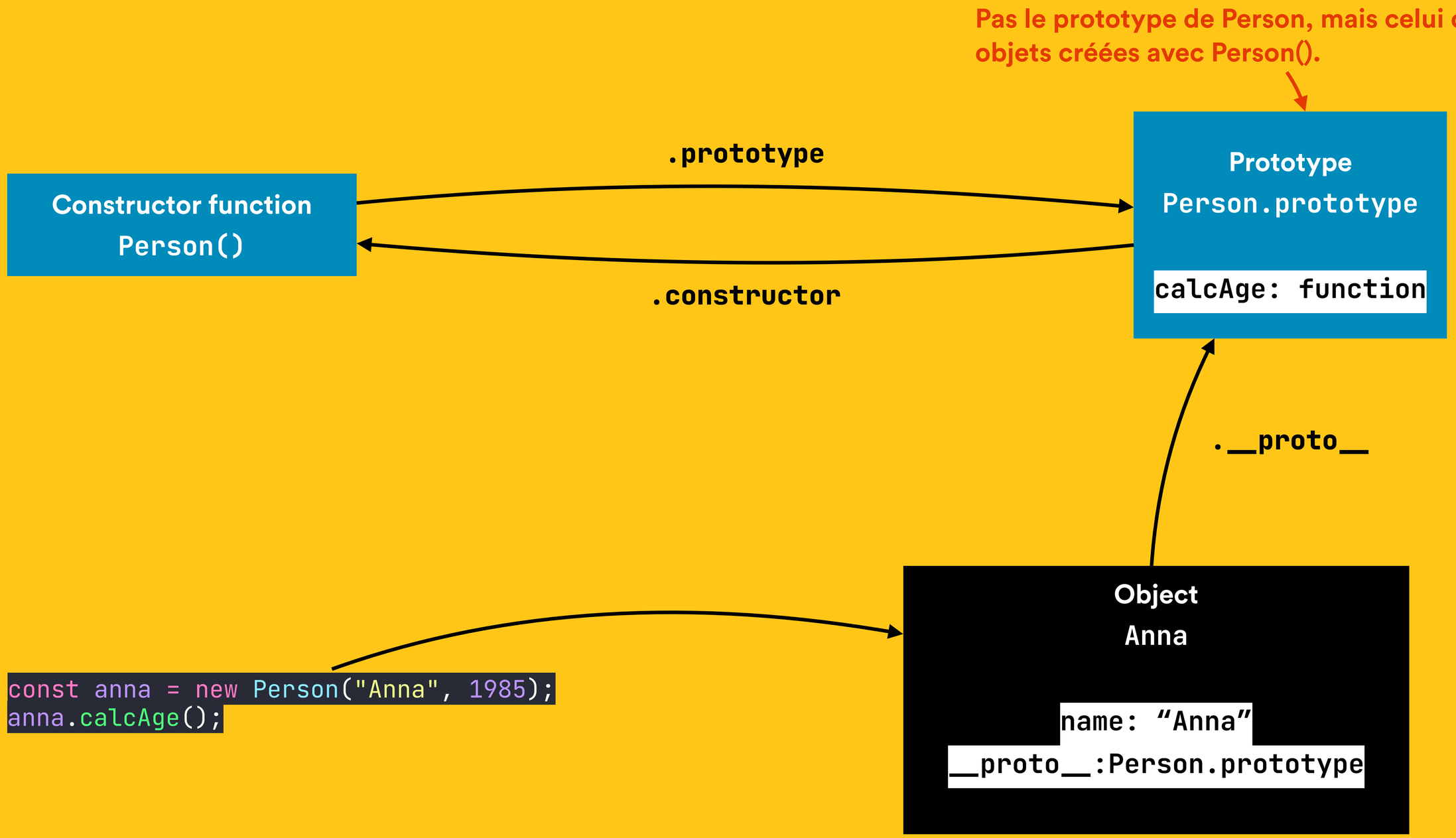

```
const anna = new Person("Anna", 1985);
anna.calcAge();
```

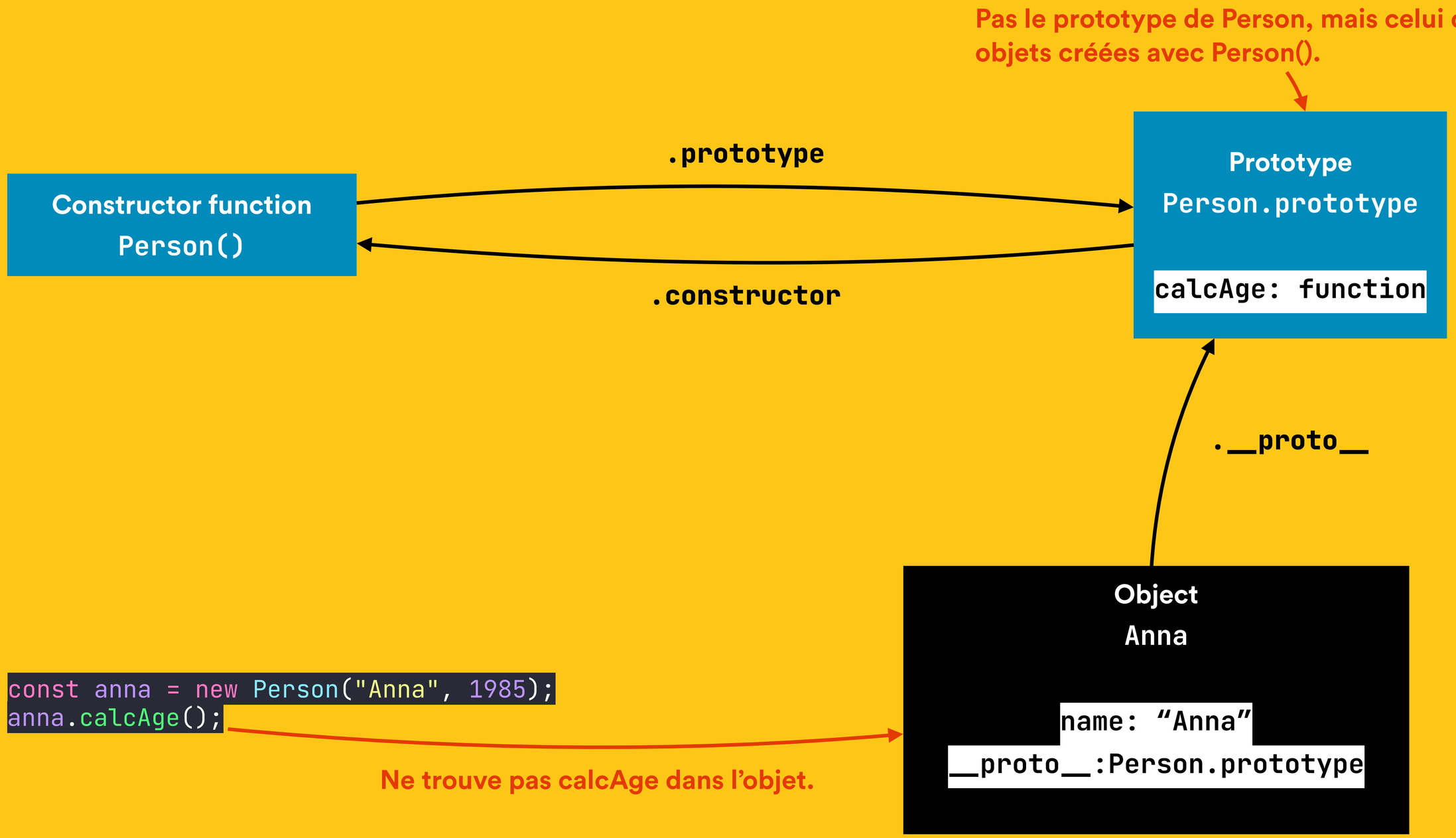
Pas le prototype de Person, mais celui des objets créées avec Person().

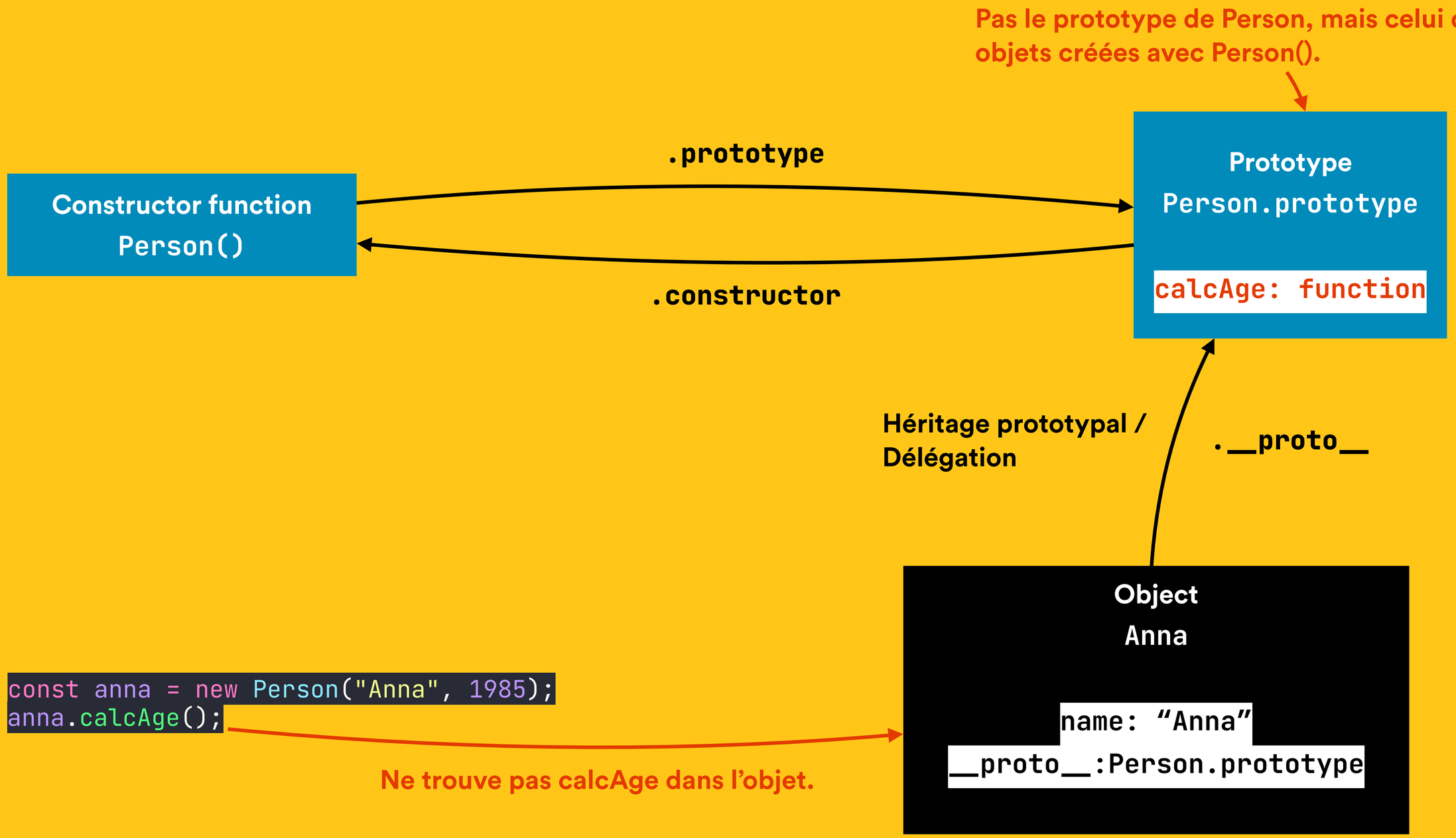



```
const anna = new Person("Anna", 1985);
anna.calcAge();
```

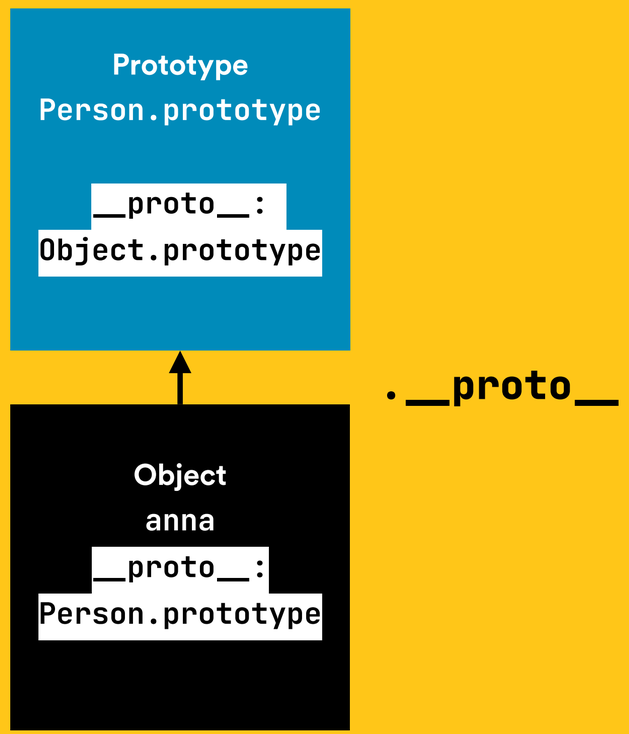


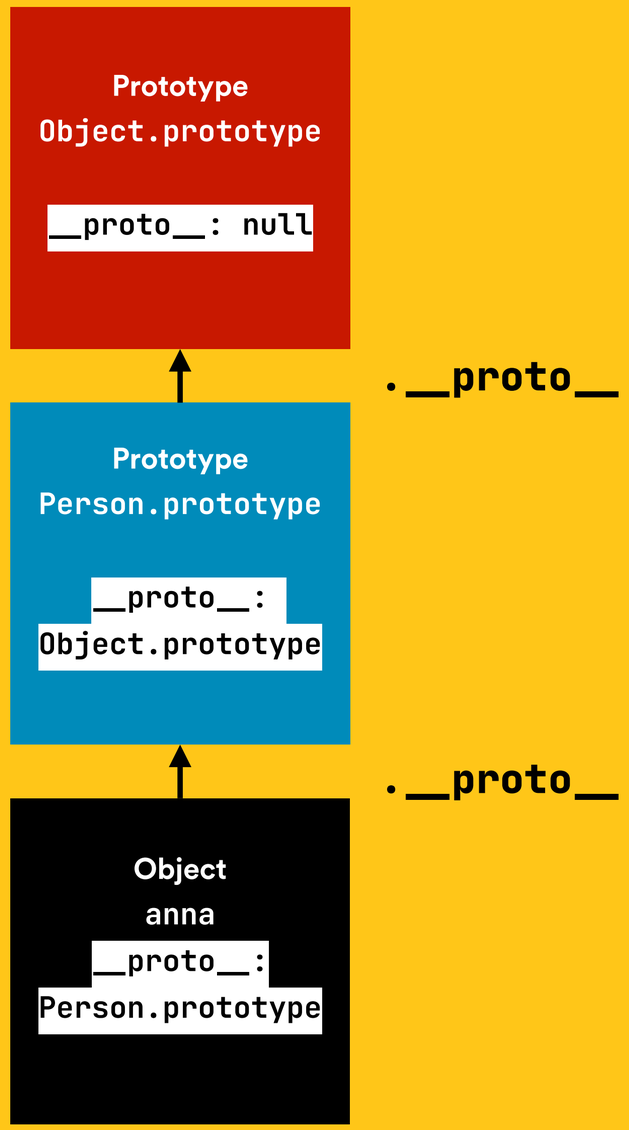

Pas le prototype de Person, mais celui des objets créées avec Person().
Constructor function
Person()
.prototype
.constructor
Prototype
Person.prototype
calcAge: function
.__proto__
Object
Anna
name: “Anna”
__proto__:Person.prototype
const anna = new Person("Anna", 1985);
anna.calcAge();

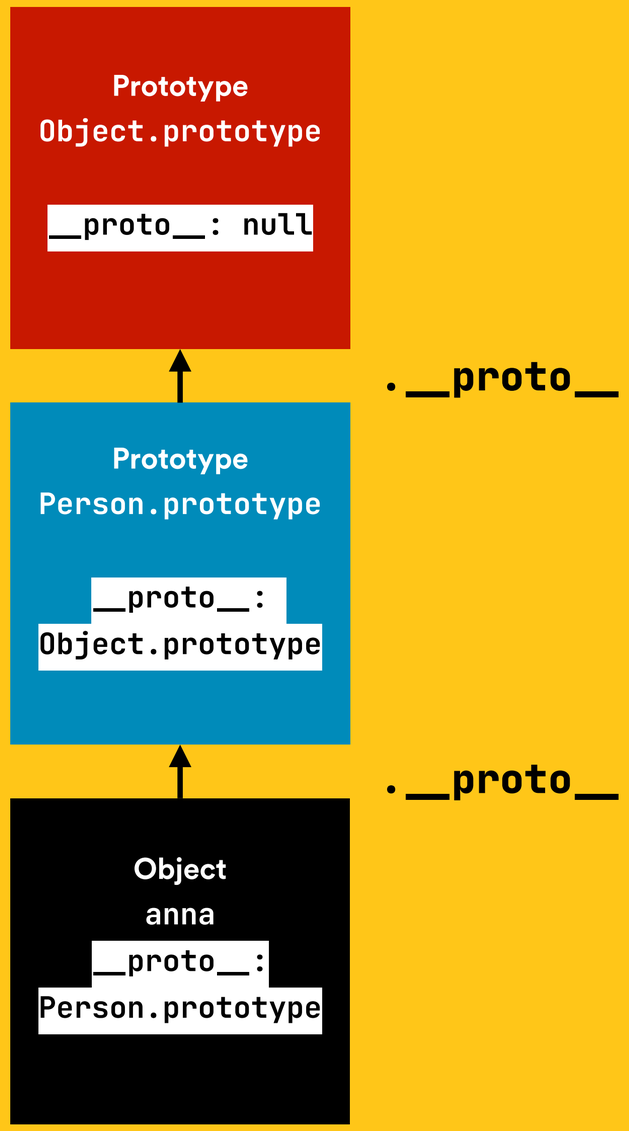

Pas le prototype de Person, mais celui des objets créées avec Person().
Constructor function
Person()
.prototype
.constructor
Prototype
Person.prototype
calcAge: function
.__proto__
Object
Anna
name: “Anna”
__proto__:Person.prototype
const anna = new Person("Anna", 1985);
anna.calcAge();
Ne trouve pas calcAge dans l’objet.


Pas le prototype de Person, mais celui des objets créées avec Person().
Constructor function
Person()
.prototype
.constructor
Prototype
Person.prototype
calcAge: function
Héritage prototypal / Délégation
.__proto__
Object
Anna
name: “Anna”
__proto__:Person.prototype
const anna = new Person("Anna", 1985);
anna.calcAge();
Ne trouve pas calcAge dans l’objet.

Object
anna
__proto__:
Person.prototype


Prototype
Person.prototype
__proto__:
Object.prototype
.__proto__
Object
anna
__proto__:
Person.prototype


Prototype
Object.prototype
__proto__: null
.__proto__
Prototype
Person.prototype
__proto__:
Object.prototype
.__proto__
Object
anna
__proto__:
Person.prototype



Ceci est la chaîne de Prototypes

Une série de liens entre objets, liés au travers de Prototypes.

L'héritage prototypal se fait au travers de la chaîne, jusque'à `null`.